

- Have more than 1 hidden layer.
- With ReLU both shallow & deep networks describe piecewise linear mappings from input to output.
- Shallow networks can describe arbitrarily complex functions, with enough hidden units.
↳ BUT, for some functions the # of hidden units is impractically large.
- ↳ Deep networks make linear regions than shallow networks for a given number of parameters.
- ↳ They describe a broader family of functions.

4.1: Composing Neural Networks:

- Consider 2 shallow networks where the output of one is the input to the second:
* 3 hidden units each:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

$$\text{and: } y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

→ First network

The second network takes y as input & returns y' . Its defined as:

$$h'_1 = a[\theta'_{10} + \theta'_{11}y]$$

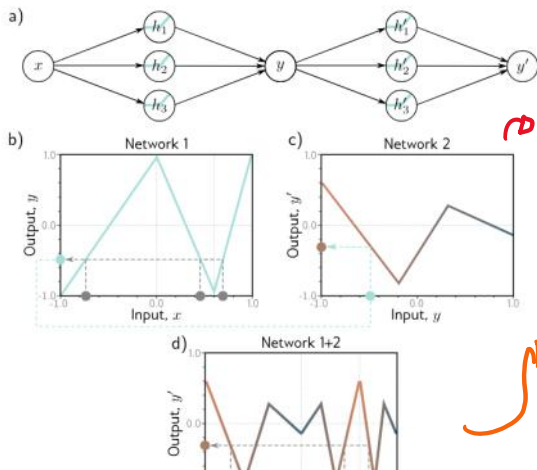
$$h'_2 = a[\theta'_{20} + \theta'_{21}y]$$

$$h'_3 = a[\theta'_{30} + \theta'_{31}y]$$

→ Second network

$$\text{and: } y' = \phi'_0 + \phi'_1 h'_1 + \phi'_2 h'_2 + \phi'_3 h'_3$$

- With ReLU, this model describes a family of piecewise linear functions.
- This network gives potentially more linear regions than a shallow network with 6 hidden units.



↳ Sign of the slopes of the first network impact if the second network is flipped or not. The magnitude impacts how compressed it is. (+ slope = normal, - slope = flipped), (higher magnitude = more compression)

↳ When composed, it duplicates the function produced by network 2 into linear pieces! Scaled by

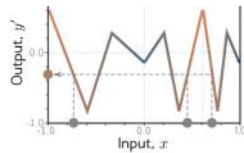


Figure 4.1 Composing two single-layer networks with three hidden units each. a) The output y of the first network constitutes the input to the second network. b) The first network maps inputs $x \in [-1, 1]$ to outputs $y \in [-1, 1]$ using a function comprising three linear regions that are chosen so that they alternate the sign of their slope (fourth linear region is outside range of graph). Multiple inputs x (gray circles) now map to the same output y (cyan circle). c) The second network defines a function comprising three linear regions that takes y and returns y' (i.e., the cyan circle is mapped to the brown circle). d) The combined effect of these two functions when composed is that (i) three different inputs x are mapped to any given value of y by the first network and (ii) are processed in the same way by the second network; the result is that the function defined by the second network in panel (c) is duplicated three times, variously flipped and rescaled according to the slope of the regions of panel (b). (Interactive figure)

Function produced by network 2
three times, flipped & scaled by
the slope of the linear regions
of network 1.

• Since it gets duplicated 3 times, it produces 9 linear regions.

↳ The same principle gets applied in higher dimensions.

• Can be thought of as: the first network "folds" the input space x back onto itself, so that multiple inputs generate the same output.

The second network applies a function, which is replicated at all points that were folded on top of one another.

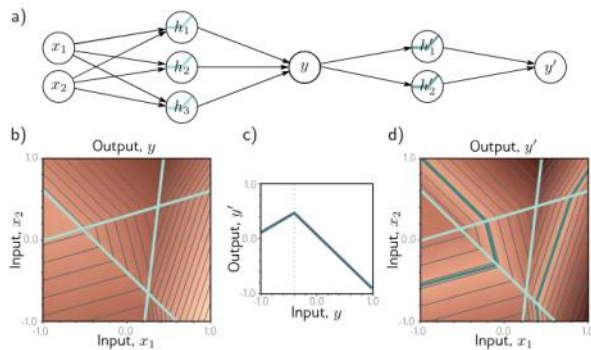


Figure 4.2 Composing neural networks with a 2D input. a) The first network (from figure 3.8) has three hidden units and takes two inputs x_1 and x_2 and returns a scalar output y . This is passed into a second network with two hidden units to produce y' . b) The first network produces a function consisting of seven linear regions, one of which is flat. c) The second network defines a function comprising two linear regions in $y \in [-1, 1]$. d) When these networks are composed, each of the six non-flat regions from the first network is divided into two new regions by the second network to create a total of 13 linear regions.

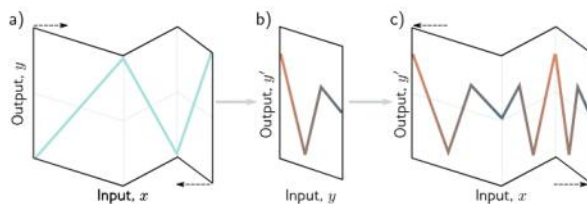


Figure 4.3 Deep networks as folding input space. a) One way to think about the first network from figure 4.1 is that it "folds" the input space back on top of itself. b) The second network applies its function to the folded space. c) The final output is revealed by "unfolding" again.

4.2: From Composing Networks to Deep Networks:

• The above is a special case of a deep network with two hidden layers.

• The output of the first network is a linear combination of the activations at the hidden units:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

$$\hookrightarrow y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

- The first operations of the second network are linear in the output of the first network:

$$\hookrightarrow \theta'_{10} + \theta'_{11} y, \theta'_{20} + \theta'_{21} y, \theta'_{30} + \theta'_{31} y$$

- Applying one linear function to another yields another linear function:

$$\hookrightarrow h'_1 = a[\theta'_{10} + \theta'_{11} y]$$

$$h'_2 = a[\theta'_{20} + \theta'_{21} y]$$

$$h'_3 = a[\theta'_{30} + \theta'_{31} y]$$

* Substituting $y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$

$$h'_1 = a[\theta'_{10} + \theta'_{11} \phi_0 + \theta'_{11} \phi_1 h_1 + \theta'_{11} \phi_2 h_2 + \theta'_{11} \phi_3 h_3]$$

$$h'_2 = a[\theta'_{20} + \theta'_{21} \phi_0 + \theta'_{21} \phi_1 h_1 + \theta'_{21} \phi_2 h_2 + \theta'_{21} \phi_3 h_3]$$

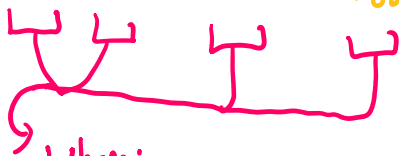
$$h'_3 = a[\theta'_{30} + \theta'_{31} \phi_0 + \theta'_{31} \phi_1 h_1 + \theta'_{31} \phi_2 h_2 + \theta'_{31} \phi_3 h_3]$$

* Which can be written as:

$$h'_1 = a[\psi_{10} + \psi_{11} h_1 + \psi_{12} h_2 + \psi_{13} h_3]$$

$$h'_2 = a[\psi_{20} + \psi_{21} h_1 + \psi_{22} h_2 + \psi_{23} h_3]$$

$$h'_3 = a[\psi_{30} + \psi_{31} h_1 + \psi_{32} h_2 + \psi_{33} h_3]$$



Where:

$$\psi_{x0} = \theta'_{x0} + \theta'_{x1} \phi_0$$

$$\psi_{x1} = \theta'_{x1} \phi_1$$

$$\psi_{x2} = \theta'_{x1} \phi_2$$

$$\psi_{x3} = \theta'_{x1} \phi_3$$

- This results in a network with 2 hidden layers.

$x_0 = 1, x_1, x_3$

- This results in a network with 2 hidden layers:

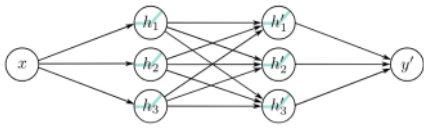


Figure 4.4 Neural network with one input, one output, and two hidden layers, each containing three hidden units.

- A network with two layers can represent the family of functions created by passing the output of one single layer network to another.

↳ And broader!

4.3: Deep Neural Networks:

- We now consider the general case of a deep network with 2 hidden layers, each layer has 3 hidden units.

* First layer:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

* Second layer:

$$h'_1 = a[\gamma_{10} + \gamma_{11}h_1 + \gamma_{12}h_2 + \gamma_{13}h_3]$$

$$h'_2 = a[\gamma_{20} + \gamma_{21}h_1 + \gamma_{22}h_2 + \gamma_{23}h_3]$$

$$h'_3 = a[\gamma_{30} + \gamma_{31}h_1 + \gamma_{32}h_2 + \gamma_{33}h_3]$$

* And the output:

$$y' = \phi_0' + \phi_1'h'_1 + \phi_2'h'_2 + \phi_3'h'_3$$

• These equations give us another way to think about deep networks:

1) h_1, h_2, h_3 in the first layer are computed by forming linear functions of the input x and passing them through ReLU activation functions.

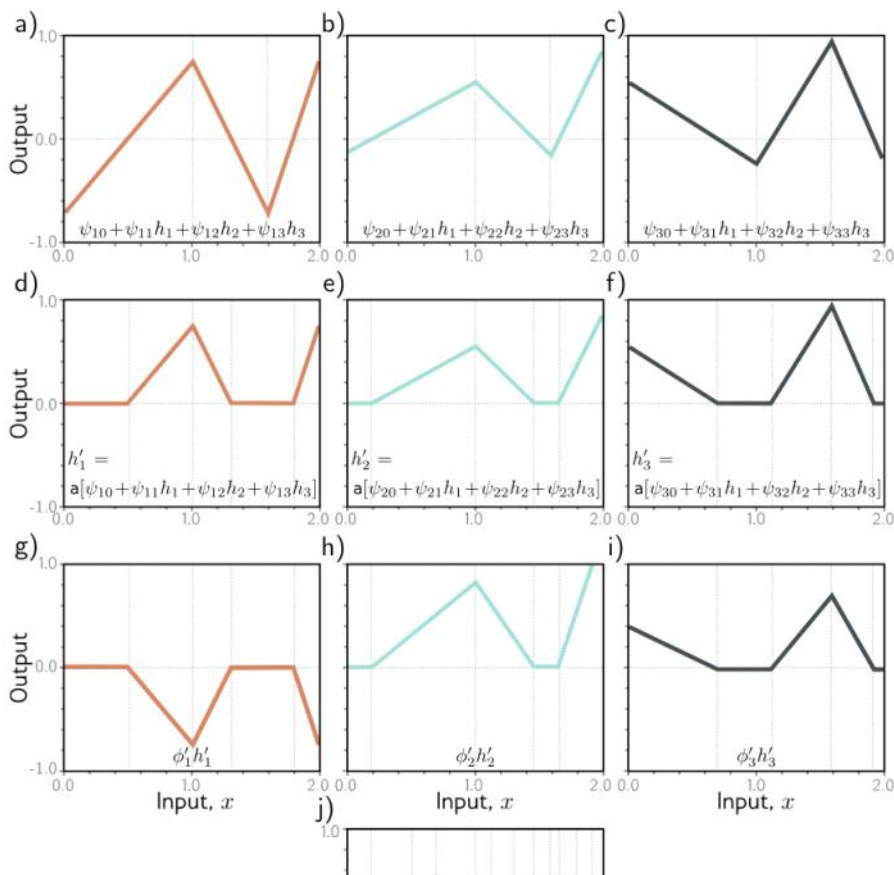
2) The pre-activations at the second layer are computed by taking three new linear functions of the hidden units h_1, h_2, h_3 .

At this point, we effectively have a shallow network with 3 outputs.

↳ 3 piecewise linear functions with "joints" between linear regions, in the same places.

3) At the second hidden layer, another ReLU function is applied, which clips each function, which adds new "joints" to each.

4) The final output is a linear combination of these hidden units.



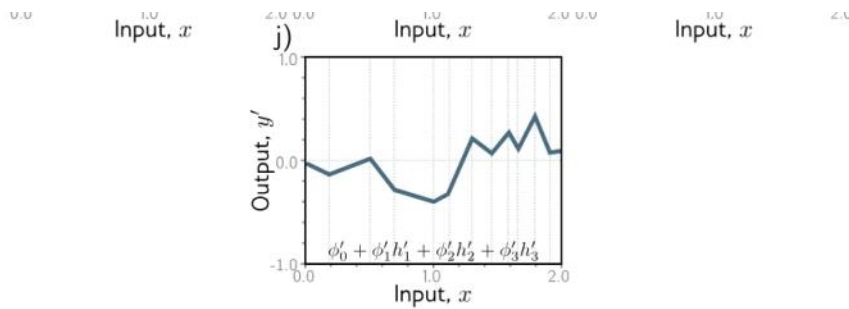


Figure 4.5 Computation for the deep network in figure 4.4. a-c) The inputs to the second hidden layer (i.e., the pre-activations) are three piecewise linear functions where the “joints” between the linear regions are at the same places (see figure 3.6). d-f) Each piecewise linear function is clipped to zero by the ReLU activation function. g-i) These clipped functions are then weighted with parameters ϕ'_1, ϕ'_2 , and ϕ'_3 , respectively. j) Finally, the clipped and weighted functions are summed and an offset ϕ'_0 that controls the overall height is added. (Interactive figure)

- So, we can think of each layer as “folding” the input space OR as creating new functions, which are clipped [thus creating new regions], then recombined.

↳ The 1st View emphasizes the dependencies in the output function, but not how clipping makes new joints.

↳ The 2nd View is the opposite.

↳ Both provide only partial insight in how deep networks work.

- At the end of the day, it's just an equation relating input x to output y' :

$$y' = \phi'_0 + \phi'_1 a[\psi_{10} + \psi_{11}a[\theta_{10} + \theta_{11}x] + \psi_{12}a[\theta_{20} + \theta_{21}x] + \psi_{13}a[\theta_{30} + \theta_{31}x]] + \phi'_2 a[\psi_{20} + \psi_{21}a[\theta_{10} + \theta_{11}x] + \psi_{22}a[\theta_{20} + \theta_{21}x] + \psi_{23}a[\theta_{30} + \theta_{31}x]] + \phi'_3 a[\psi_{30} + \psi_{31}a[\theta_{10} + \theta_{11}x] + \psi_{32}a[\theta_{20} + \theta_{21}x] + \psi_{33}a[\theta_{30} + \theta_{31}x]], \quad (4.10)$$

↳ But this is difficult to understand.

HyperParameters:

- We can extend deep networks to more than 2 hidden layers.
- The # of hidden units in each layer is the width of the network.
- The # of hidden layers is the depth.
- The total # of hidden units is a measure of the network's capacity.

- The total # of hidden units is a measure of the network's capacity.
- We denote the # of layers as K, & the number of hidden units in each layer as $D_1, D_2, \dots, D_K$.
- These are all hyperparameters.
 - ↳ Values chosen before training the model.
 - ↳ Describe architecture choices, training choices (LR).
- For fixed hyperparameters, the model describes a family of functions.
 - ↳ The parameters specify the exact function.
- With hyperparameters, we can think of neural networks as representing a family of families of functions relating input to output.

4.4: Matrix Notation:

- We can describe the equations above with matrix notation:

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = \bar{a} \left[\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x \right]$$

$$\begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix} = \bar{a} \left[\begin{bmatrix} \psi_{10} \\ \psi_{20} \\ \psi_{30} \end{bmatrix} + \begin{bmatrix} \psi_{11} & \psi_{12} & \psi_{13} \\ \psi_{21} & \psi_{22} & \psi_{23} \\ \psi_{31} & \psi_{32} & \psi_{33} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} \right]$$

$$y' = \phi'_0 + [\phi'_1 \ \phi'_2 \ \phi'_3] \begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix}$$

or even more compactly as:

$$\bar{h} = \bar{a} [\bar{\theta}_0 + \bar{\theta} x]$$

$$\bar{h}' = \bar{a} [\bar{\psi}_0 + \bar{\Psi} \bar{h}]$$

$$y' = \phi'_0 + \bar{\phi}' \bar{h}'$$

In each case, $\bar{a}[\cdot]$ applies the activation function separately to every element of its vector input.

General Formulation:

- With many layers, the notation gets tough.
- So we describe the vector of hidden units at layer K as $\bar{h}_K$, the vector of biases that contribute to layer $K+1$ as $\bar{b}_K$, the weights that are applied to the K th layer contribute to the $(K+1)^{\text{th}}$ layer as $\bar{\Omega}_K$.

- A general deep network $\bar{y} = \bar{f}[\bar{x}, \bar{\theta}]$ with K layers is written as:

$$\bar{h}_1 = \bar{a}[\bar{b}_0 + \bar{\Omega}_0 \bar{x}]$$

$$\bar{h}_2 = \bar{a}[\bar{b}_1 + \bar{\Omega}_1 \bar{h}_1]$$

$$\bar{h}_3 = \bar{a}[\bar{b}_2 + \bar{\Omega}_2 \bar{h}_2]$$

$$\bar{h}_3 = \bar{a} [\bar{B}_2 + \bar{\Omega}_2 \bar{h}_2]$$

$\vdots$

$$\bar{h}_K = \bar{a} [\bar{B}_{K-1} + \bar{\Omega}_{K-1} \bar{h}_{K-1}]$$

$$\bar{y} = \bar{B}_K + \bar{\Omega}_K \bar{h}_K$$

- The parameters $\emptyset$ of the model are all these weight matrices & bias vectors:

$$\emptyset = \{ \bar{B}_k, \bar{\Omega}_k \}_{k=0}^K$$

- If the k^{th} layer has D_k hidden units, then the bias vector $\bar{B}_{k-1}$ will be of size D_k .
- The last bias vector $\bar{B}_K$ has the size D_0 of the output
- The first weight matrix $\bar{\Omega}_0$ has size $D_1 \times D_i$, where D_i is the size of the input.
- The last weight matrix $\bar{\Omega}_K$ is $D_0 \times D_K$
- The remaining matrices $\bar{\Omega}_k$ are $D_{k+1} \times D_k$.

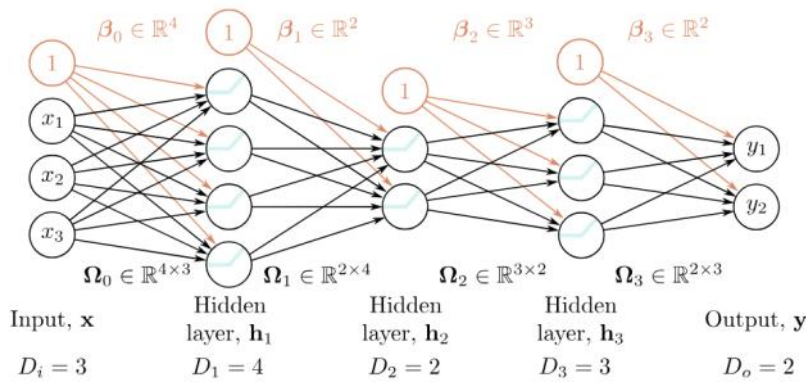


Figure 4.6 Matrix notation for network with $D_i = 3$ -dimensional input x , $D_o = 2$ -dimensional output y , and $K = 3$ hidden layers h_1, h_2 , and h_3 of dimensions $D_1 = 4$, $D_2 = 2$, and $D_3 = 3$ respectively. The weights are stored in matrices Ω_k that multiply the activations from the preceding layer to create the pre-activations at the subsequent layer. For example, the weight matrix Ω_1 that computes the pre-activations at h_2 from the activations at h_1 has dimension 2×4 . It is applied to the four hidden units in layer one and creates the inputs to the two hidden units at layer two. The biases are stored in vectors β_k and have the dimension of the layer into which they feed. For example, the bias vector β_2 is length three because layer h_3 contains three hidden units.

- We can equivalently write the network as a single function:

$$y = \beta_K + \Omega_K a [\beta_{K-1} + \Omega_{K-1} a [\dots \beta_2 + \Omega_2 a [\beta_1 + \Omega_1 a [\beta_0 + \Omega_0 x] \dots]]]$$

4.5: Shallow vs. Deep Neural Networks:

Ability to approximate different functions:

- We know a shallow network with enough capacity could model any continuous function arbitrarily close.
- We saw that a deep net with 2 hidden layers can represent the composition of 2 shallow nets. If the 2nd network computes the identity function, then the deep net replicates the shallow net.
- So it can also approximate any continuous function arbitrarily close given enough capacity.

Number of linear regions per parameter:

• A shallow net with n hidden units can create at most $n+1$ linear regions.

Number of linear regions per parameter:

- A shallow net with 1 input, 1 output, $\geq D \geq 2$ hidden units can create up to $D+1$ linear regions; has $3D+1$ parameters.
- A deep net with 1 input, 1 output, $\geq K$ layers of $D \geq 2$ hidden units can create a function up to $(D+1)^K$ linear regions using $3D+1 + (K-1)D(D+1)$ parameters.

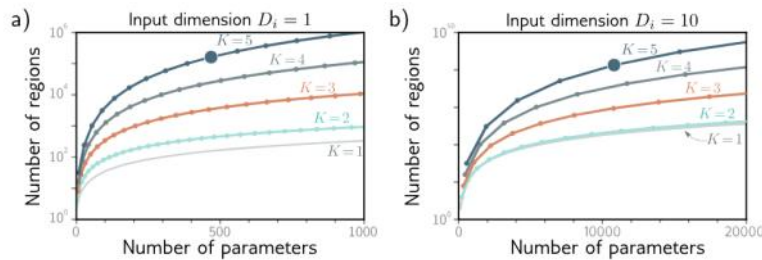


Figure 4.7 The maximum number of linear regions for neural networks increases rapidly with the network depth. a) Network with $D_i = 1$ input. Each curve represents a fixed number of hidden layers K , as we vary the number of hidden units D per layer. For a fixed parameter budget (horizontal position), deeper networks produce more linear regions than shallower ones. A network with $K = 5$ layers and $D = 10$ hidden units per layer has 471 parameters (highlighted point) and can produce 161,051 regions. b) Network with $D_i = 10$ inputs. Each subsequent point along a curve represents ten hidden units. Here, a model with $K = 5$ layers and $D = 50$ hidden units per layer has 10,801 parameters (highlighted point) and can create more than 10^{40} linear regions.

- Deep nets create much more complex functions for a fixed parameter budget.
- Seems nice, but these regions contain complex dependencies; symmetry (the folding analogy)
 - ↳ Not clear if greater number of regions is an advantage, unless:
 - 1) There are similar symmetries in the real world functions we want to approximate.
 - or 2) There is evidence that the input-output mapping involves a combination of simpler functions.

Depth Efficiency:

- A deep network can efficiently approximate some functions that a shallow network requires an exponential amount of hidden units.
- Called the depth efficiency of neural networks.

Large, Structured inputs:

- 1. how soon DNNs changed networks to be

Image Structure Models

- We've seen fully connected networks so far.
- These are NOT practical for large, structured outputs like images.
 - ↳ Inputs can be $\sim 10^6$ pixels
- # of parameters would be prohibitive
- No point in independently learning to recognize the same object at every part of the image [i.e., different parts of the image should be processed similarly]
- Solution: Process local image regions in parallel, then integrate information from increasingly large regions.
 - ↳ Hard to do without using multiple layers.

Training & Generalization

- Usually easier to train moderately deep networks than shallow ones.
- Could be that overparametrized models have a large family of roughly equivalent solutions that are easy to find.
 - ↳ More params than training examples
- But as we add more layers, training gets tough again.
- Deep nets seem to generalize to new data better than shallow ones.

* These phenomena are not well understood.

4.6: Summary:

- When composing two shallow networks, the first "folds" the input space, & the second applies a piecewise linear function.
 - ↳ The effect of the 2nd network are distributed across the input space in a way that is difficult to understand.

second utilizes a piecewise linear function.

↳ The effect of the 2nd network are duplicated where the input space is folded onto itself.

- This composition is a special case of a deep network with 2 layers.

↳ ReLU in each layer clips the input functions in multiple places
; makes more "joints" in the output function.

- Hyperparameters comprise the # of hidden layers / hidden units.

- Compared shallow ; deep nets:

↳ Both approximate any function with enough capacity

↳ Deep nets make many more linear regions per parameter.

↳ Some functions are more efficient to approximate with deep nets.

↳ Large, structured inputs are best processed in multiple stages

↳ Practice shows deep nets give better results for most tasks.